

Volume 04

WarrantyGraph

Ontology · RDF · OWL · Turtle

Formal schema, serializations, Neo4j mapping, export tooling

Full project library

This multi-volume set covers the entire repository — not a single session. Algorithms, theory, annotated code, RDF/OWL, pipelines, and deploy are all included.

1. Concepts (beginner → precise)

Term	Plain English	In this project
Ontology	Dictionary of types + allowed links	Product, Symptom, FM, Part...
RDF	Data as subject–predicate–object triples	Export format / interchange
RDFS	Classes + domain/range basics	Used in Turtle headers
OWL	Richer ontology language	owl:Class, ObjectProperty, Ontology header
Turtle (.ttl)	Readable RDF text syntax	docs/ontology/*.ttl
RDF/XML (.owl)	XML serialization of RDF/OWL	warranty-diagnosis.owl
TBox	Schema layer	Classes & properties
ABox	Instance layer	wm-001, specific symptoms...
Neo4j graph	Operational store for Cypher	Runtime truth for diagnosis

Why both Neo4j and RDF?

Neo4j is optimized for multi-hop runtime diagnosis. RDF/OWL is the formal interchange / documentation surface for semantic-web tools and enterprise ontology governance. Export with: `python -m graph.rdf_ontology_export`

2. Class model (OWL TBox)

Classes: Product, Model, SKU, Asset, Symptom, ErrorCode, FailureMode, DiagnosticStep, Component, Part, Claim, HistoricalResolution, WarrantyPolicy.

```
@prefix wd:    <https://example.org/warranty-diagnosis#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs:  <http://www.w3.org/2000/01/rdf-schema#> .

<https://example.org/warranty-diagnosis/ontology> a owl:Ontology ;
  rdfs:label "Enterprise Warranty Diagnosis Ontology" ;
  owl:versionInfo "1.0.0" .

wd:Product a owl:Class ;
  rdfs:label "Product" ;
  rdfs:comment "A manufacturable product family / catalog product." .

wd:FailureMode a owl:Class ;
  rdfs:comment "Diagnosed failure mode (FMEA-aligned)." .

wd:indicates a owl:ObjectProperty ;
  rdfs:domain wd:Symptom ;
  rdfs:range wd:FailureMode ;
  rdfs:comment "Symptom indicates failure mode. Neo4j: INDICATES." .
```

What the code/export does: declares machine-readable types. **Why:** tools and humans share one vocabulary; maps 1:1 to Neo4j labels/rel types.

3. Object properties ↔ Neo4j relationships

OWL property	Neo4j type	Meaning
wd:hasSymptom	HAS_SYMPTOM	Product presents symptom
wd:indicates	INDICATES	Symptom→FM with confidence
wd:requiresPart	REQUIRES_PART	FM needs part
wd:impactsComponent	IMPACTS_COMPONENT	FM hits BOM subsystem
wd:realizedBy	REALIZED_BY	Component→Part
wd:nextStep	NEXT_STEP	Diagnostic tree branch
wd:confirms / rulesOut	CONFIRMS / RULES_OUT	Step vs FM
wd:instanceOf	INSTANCE_OF	Asset→Product
wd:compatibleWith	COMPATIBLE_WITH	SKU→Part

4. Instance (ABox) Turtle example

```

wd:product_wm-001 a wd:Product ;
  wd:productId "wm-001" ;
  wd:name "AquaHome Front Load 8kg" ;
  wd:hasSymptom wd:symptom_wm-s03 ;
  wd:canHave wd:fm_wm-fm02 .

wd:symptom_wm-s03 a wd:Symptom ;
  wd:description "Will not drain / E21" ;
  wd:severity "high" ;
  wd:indicates wd:fm_wm-fm02 .

wd:fm_wm-fm02 a wd:FailureMode ;
  rdfs:label "Drain pump failure" ;
  wd:impactsComponent wd:component_wm-c02 ;
  wd:requiresPart wd:part_wm-p02 .

wd:component_wm-c02 a wd:Component ;
  wd:subsystem "Plumbing" ;
  wd:realizedBy wd:part_wm-p02 .

```

Confidence on edges

Neo4j stores confidence on INDICATES relationships. In OWL export we reify with owl:Axiom + wd:confidence annotation so the triple can carry a literal weight.

5. RDF/XML (OWL) fragment

```

<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:owl="http://www.w3.org/2002/07/owl#"
  xmlns:wd="https://example.org/warranty-diagnosis#">
  <owl:Ontology rdf:about="https://example.org/warranty-diagnosis/ontology"/>
  <owl:Class rdf:about="https://example.org/warranty-diagnosis#Product"/>
  <owl:ObjectProperty rdf:about="https://example.org/warranty-diagnosis#indicates">
    <rdfs:domain rdf:resource="...#Symptom"/>
    <rdfs:range rdf:resource="...#FailureMode"/>
  </owl:ObjectProperty>
</rdf:RDF>

```

Same triples as Turtle, different syntax. Some enterprise tools prefer RDF/XML (.owl files).

6. Export tooling (code that writes RDF)

File: graph/rdf_ontology_export.py

```

# WHAT: build TBox triples + optional ABox from catalog JSON
# WHY: stdlib-only Turtle/RDF/XML without requiring rdflib at runtime
python -m graph.rdf_ontology_export --schema-only \
  --out docs/ontology/warranty-diagnosis-schema.ttl
python -m graph.rdf_ontology_export --product-id wm-001 \
  --out docs/ontology/wm-001.ttl
python -m graph.rdf_ontology_export --format rdxml \
  --out docs/ontology/warranty-diagnosis.owl

```

- schema_triples_ttl() — classes + object/datatype properties
- instance_ttl_for_product() — one product neighborhood as individuals
- catalog_to_turtle() — full/filtered ABox + assets/claims/policies
- schema_to_rdxml() — OWL/XML document

7. Live schema excerpt from repo

```

@prefix wd: <https://example.org/warranty-diagnosis#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix prov: <http://www.w3.org/ns/prov#> .
<https://example.org/warranty-diagnosis/ontology> a owl:Ontology ;
  rdfs:label "Enterprise Warranty Diagnosis Ontology" ;
  rdfs:comment "Formal schema for product, symptom, failure mode, BOM component, parts, diagnostic steps, assets, and claims. Product structure (Component/Part) implements the industrial hierarchy aspect often called topology in ISO 14224 / IEC 81346 product-aspect terms – not a separate subsystem." ;
  owl:versionInfo "1.0.0" .

wd:Product a owl:Class ;
  rdfs:label "Product" ;
  rdfs:comment "A manufacturable product family / catalog product." .

wd:Model a owl:Class ;
  rdfs:label "Model" ;
  rdfs:comment "Engineering model under a product family." .

wd:SKU a owl:Class ;
  rdfs:label "SKU" ;
  rdfs:comment "Sellable stock-keeping unit / revision of a model." .

wd:Asset a owl:Class ;
  rdfs:label "Asset" ;
  rdfs:comment "Installed unit (serial) bound to a product/SKU." .

wd:Symptom a owl:Class ;
  rdfs:label "Symptom" ;
  rdfs:comment "Customer-observable or technician-observed symptom." .

wd:ErrorCode a owl:Class ;
  rdfs:label "ErrorCode" ;
  rdfs:comment "Device-reported diagnostic code." .

wd:FailureMode a owl:Class ;
  rdfs:label "FailureMode" ;
  rdfs:comment "Diagnosed failure mode (FMEA-aligned)." .

wd:DiagnosticStep a owl:Class ;
  rdfs:label "DiagnosticStep" ;
  rdfs:comment "Troubleshooting or confirmation step." .

wd:Component a owl:Class ;
  rdfs:label "Component" ;
  rdfs:comment "BOM / subsystem component (product structure)." .

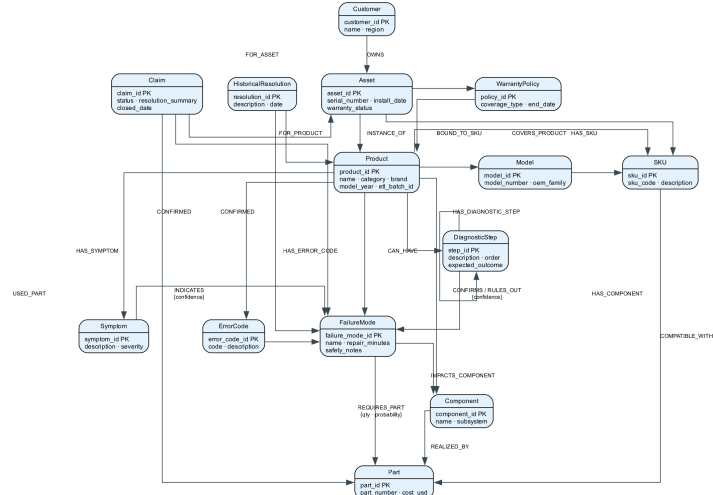
wd:Part a owl:Class ;
  rdfs:label "Part" ;
  rdfs:comment "Serviceable replacement part." .

wd:Claim a owl:Class ;
  rdfs:label "Claim" ;
  rdfs:comment "Warranty claim with confirmed failure and parts used." .

```

8. Standards alignment

Standard	Relevance
W3C RDF 1.1 / RDFS / OWL	Serialization & ontology language
W3C PROV-O	Provenance fields on entities
ISO 14224	Equipment hierarchy + failure taxonomy inspiration
ISO/IEC 81346	Product-aspect structure ↔ Component/BOM
FMEA / AIAG-VDA	FailureMode scoring semantics



ERD = property-graph view of the same ontology

9. Topology note (do not confuse)

Industrial **product structure** (BOM components) is modeled **inside** this ontology (Component, IMPACTS_COMPONENT, REALIZED_BY). We do **not** maintain a separate Topology subsystem. W3C BOT (Building Topology Ontology) is for buildings — out of scope.